🏄 Especificação de Execução End-to-End no Surfpool & Upgrades Pós-Stocklana para o ProofGate / ValueCurator

> ***Colosseum Crypto World's Fair 2026 / Eternal / Accelerator & Ecossistema de Grants Descentralizadas***
> ***Documento de Engenharia & Produto***: *Implementação Prática de Todos os Upgrades Identificados no Post-Mortem do Stocklana*
> ***Padrões de Referência Incorporados***:
>
> - ***Roster Finance*** *(1º Lugar Stocklana — US$ 35k): Surfpool Mainnet Forking, Invariância a Splits via Token-2022* `ScaledUiAmount`*, Proteção de 75h de Fim de Semana e Seção "What is Live and What is Not"*
> - ***Draw*** *(2º Lugar Stocklana — US$ 20k): Execução Atômica contra Reservas Reais em Fork da Mainnet e Gasless Fee-Payer para Carteiras com Zero SOL*
> - ***CRESCO*** *(4º Lugar Stocklana — US$ 15k): Roteiro Canônico em 8 Passos, Exceção de Uso Único Vinculada a Nonce, Recusa de Adulteração e Anti-Replay*
> - ***Xorr Finance*** *(5º Lugar Stocklana — US$ 15k): Off-Hours Guard (* `Equity.US` *vs. Pool DEX), Sub-Carteiras Derivadas por Agente e Kill-Switch* `SPL Revoke` *em 1 Toque*
> - ***Sudont*** *(Vencedor Colosseum Frontier — US$ 10k): Loop ReAct de Auto-Correção do Agente (* `Fail-Closed → Self-Heal → Approved` *)*
> - ***Afterhours*** *(Vencedor Clawpump — US$ 5k): Integração Nativa* `Realms DAO Mandate → Isolated Strategy Vault`

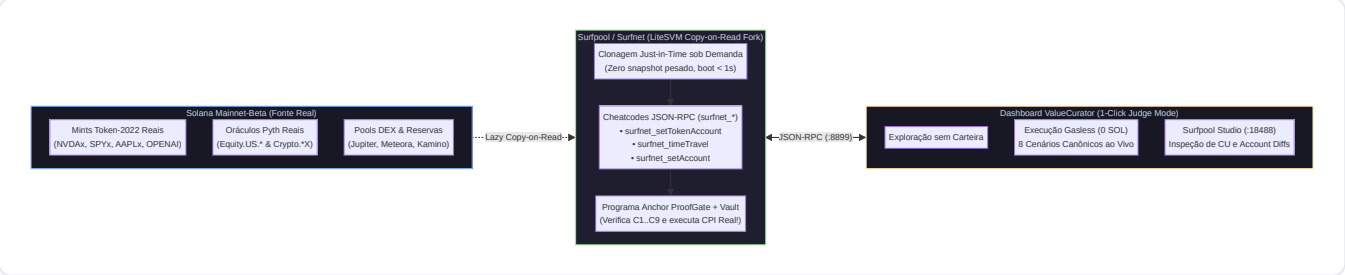
📑 Sumário Executivo da Especificação

- 🏄 **§1. Arquitetura Surfpool (** `surfpool.run` **): Como Rodar End-to-End com Estado Real da Mainnet a Custo Zero**
- ⚙️ **§2. Guia de Configuração, Runbooks IaC (** `txtx` **) e Cheatcodes JSON-RPC (** `surfnet_*` **)**
- 🛡️ **§3. Upgrade 1: Os 3 Novos Invariantes de Ações Tokenizadas e RWAs (C_7, C_8, C_9) em Rust/Anchor**
- 🔄 **§4. Upgrade 2: O Loop ReAct de Auto-Correção do Agente (** `Fail-Closed → Self-Heal → Settled` **)**
- 🎬 **§5. Upgrade 3: Roteiro Canônico de 8 Passos & "1-Click Judge Mode" (Zero-SOL, Zero-Wallet Friction)**
- 📦 **§6. Upgrade 4: Empacotamento Open-Source (** `@valuecurator/proofgate-sdk` **&** `Realms-ProofGate Adapter` **) para Grants**
- 📋 **§7. Declaração de Transparência Radical ("What is Live and What is Not")**

🏄 §1. Arquitetura Surfpool (`surfpool.run`): Como Rodar End-to-End com Estado Real da Mainnet a Custo Zero

Na análise anterior sobre a Devnet (`SOLANA_DEVNET_E2E_SYNTHETIC_RWA_ORACLE_ADAPTATION_SPEC.md`), vimos que na Devnet pura não existem pools de liquidez ativas do Jupiter para `xStocks` (`NVDAx`, `SPYx`, `AAPLx`) nem *mints* oficiais da `PreStocks` (`OPENAI`, `SPACEX`).

O **Roster Finance** (1º lugar no Stocklana) e o **Draw** (2º lugar no Stocklana) resolveram esse problema utilizando o **Surfpool** (`surfpool.run`) — o simulador de rede de alta performance (*"Anvil for Solana"*) baseado em **LiteSVM** que substitui o `solana-test-validator` com **Copy-on-Read Mainnet Forking**.



Por Que o Surfpool Elimina a Frase *"Stops Before Mainnet Execution"*?

1. **Clonagem Preguiçosa (*Lazy Copy-on-Read*)**: Quando o programa Anchor do ProofGate referencia o endereço real de um *mint* `xStock` (ex: `NVDAx` ou `SPYx`), uma conta de preço do Pyth ou uma pool da Meteora/Jupiter, o **Surfnet** busca os bytes reais da conta na Mainnet em tempo de execução e os armazena localmente em memória.
2. **Execução 100% Completa da Instrução**: A transação não para na metade. O programa Anchor verifica os 9 invariantes ($C_1 \dots C_9$) e **executa a transferência/swap real dos tokens** **`NVDAx`** / **`SPYx`** / **`USDC`**, gravando o recibo criptográfico e os diffs de contas.
3. **Um Único Build para Três Ambientes (`/api/cluster`)**: Seguindo a arquitetura do Roster Finance (*"The app builds every transaction and asks the wallet only to sign, so one build runs on fork, devnet or mainnet"*), nosso frontend expõe um seletor `/api/cluster` que alterna de forma transparente entre:
   - `surfnet-mainnet-fork` (padrão para o *1-Click Judge Demo* com ativos reais clonados da Mainnet);
   - `devnet` (usando nosso *Synthetic Peer-to-Pool Vault* no cluster público `api.devnet.solana.com`);
   - `mainnet-beta` (produção com capital real).

⚙️ §2. Guia de Configuração, Runbooks IaC (`txtx`) e Cheatcodes JSON-RPC (`surfnet_*`)

2.1. Instalação e Inicialização do Surfpool no Repositório `valuecurator`

```
# 1. Instalar o binário oficial do Surfpool
curl -sL https://run.surfpool.run/ | bash

# 2. Iniciar o Surfnet com Copy-on-Read da Mainnet e implantar automaticamente o workspace Anchor
cd /home/sukata/valuecurator/apps/web
surfpool start --network mainnet --port 8899 --studio-port 18488
```

Ao executar `surfpool start --network mainnet`, dois serviços ficam imediatamente disponíveis:

- **Endpoint RPC Compatível com Solana (`http://127.0.0.1:8899`)**: Onde o Next.js e o agente se conectam.
- **Surfpool Studio (`http://127.0.0.1:18488`)**: Painel de inspeção visual onde os juízes e auditores podem ver o consumo exato de Compute Units (CU), os logs de programas sBPF e o diff hexadecimal de cada conta antes e depois do ProofGate agir.

2.2. Programação Determinística de Cenários com Cheatcodes `surfnet_*`

O diferencial competitivo do Surfpool para a nossa bancada de testes end-to-end são os três métodos JSON-RPC nativos do namespace `surfnet_*`:

1. `surfnet_setTokenAccount` — Financiamento Instantâneo de `USDC` e `xStocks` Reais no Cofre PDA

Sem gastar 1 centavo de dólar real, creditamos `500.000 USDC` e `1.000 NVDAx` reais no PDA do cofre fiduciário do ProofGate:

```
// lib/surfpool-cheatcodes.ts
export async function seedVaultWithMainnetTokens(
  rpcUrl: string,
  vaultOwnerPda: string,
  usdcMint: string,
  nvdaxMint: string
) {
  // 1. Credita 500,000 USDC (6 decimais = 500_000_000_000) no cofre PDA
  await fetch(rpcUrl, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      jsonrpc: "2.0",
      id: "seed-usdc",
      method: "surfnet_setTokenAccount",
      params: [
        {
          owner: vaultOwnerPda,
          mint: usdcMint,
          amount: 500_000_000_000,
          state: "initialized",
        },
      ],
    }),
  });

  // 2. Credita 1,000 NVDAx (6 decimais = 1_000_000_000) para testes de venda/rebalanceamento
  await fetch(rpcUrl, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      jsonrpc: "2.0",
      id: "seed-nvdax",
      method: "surfnet_setTokenAccount",
      params: [
        {
          owner: vaultOwnerPda,
          mint: nvdaxMint,
          amount: 1_000_000_000,
          state: "initialized",
        },
      ],
    }),
  });
}
```

2. `surfnet_timeTravel` — Viagem no Tempo para Sábado às 03h10 NY (Teste do Abismo de 75 Horas)

Para provar ao vivo no vídeo de demonstração que o ProofGate protege o investidor durante a madrugada de sábado (quando a Nasdaq está fechada e ocorre uma agulhada artificial de -11% no livro do token), avançamos o relógio da blockchain via `surfnet_timeTravel`:

```
export async function travelToWeekendWickWindow(rpcUrl: string, saturday0310UnixTs: number) {
  await fetch(rpcUrl, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      jsonrpc: "2.0",
      id: "time-travel-weekend",
      method: "surfnet_timeTravel",
      params: [
        {
          absoluteTimestamp: saturday0310UnixTs, // Ex: Sábado 03:10 AM ET
        },
      ],
    }),
  });
}
```

3. `surfnet_setAccount` — Simulação de Pausa Regulatória do Emissor (`Paused`) ou Desdobramento (`Split 5x`)

Com `surfnet_setAccount`, alteramos os bytes da extensão `ScaledUiAmount` ou `Pausable` de um *mint* Token-2022 clonado da Mainnet para demonstrar ao vivo os invariantes C_8 e C_9 atuando!

🛡️ §3. Upgrade 1: Os 3 Novos Invariantes de Ações Tokenizadas e RWAs (C_7, C_8, C_9) em Rust/Anchor

Expandimos formalmente o nosso **Vetor de Invariantes Fiduciários** de 6 para **9 predicados ortogonais**:

$$\vec{\mathbf{C}}(s,a) = \big[\underbrace{C_1, C_2, C_3, C_4, C_5, C_6}_{\text{Núcleo Fiduciário Geral}}, \underbrace{C_7(\text{Off-Hours}),\ C_8(\text{ScaledUiSplit}),\ C_9(\text{MintRights})}_{\text{Núcleo Específico de Ações Tokenizadas \& RWAs}}\big] \in \{0,1\}^9$$

$$\Phi_{\text{RWA}}(s,a) = \bigwedge_{k=1}^{9} C_k(s,a)$$

3.1. Formulação Matemática de C_7, C_8 e C_9

1. C_7 **— Guarda de Base Fora de Pregão (*Off-Hours Basis & Weekend Wick Guard*)**: Seja P_{eq} o último preço oficial da ação subjacente publicado no feed `Equity.US.<TICKER>/USD` do Pyth antes do fechamento da NYSE/Nasdaq, e P_{tok} o preço atual do token on-chain (`Crypto.<TICKER>X/USD` ou cotação da pool DEX). Definimos a deriva de base (*basis drift*):

$$\Delta_{\text{basis}} = \frac{|P_{\text{tok}} - P_{\text{eq}}|}{P_{\text{eq}}}$$

Durante as 135,5 horas da semana em que o pregão regular dos EUA está fechado ($\text{Session}(t) \neq \text{REGULAR}$), o invariante C_7 impõe:

$$C_7(s,a) \iff \begin{cases} \text{True} & \text{se } \text{Session}(t) = \text{REGULAR} \\ (\Delta_{\text{basis}} \le \mathcal{M}.\delta_{\text{off_hours}}) \wedge (a.V_{\text{in}} \le \kappa_{\text{weekend}} \cdot \mathcal{M}.L_{\text{single}}) & \text{se } \text{Session}(t) \in \{\text{OFF_HOURS}, \text{WEEKEND}\} \end{cases}$$

Efeito prático: Se no sábado às 03h10 da manhã uma venda em livro raso derrubar `SPYx` ou `NVDAx` em 11% ($\Delta_{\text{basis}} = 0.11 > \delta_{\text{off_hours}} = 0.02$), o ProofGate **aborta instantaneamente** (`ERR_C7_OFF_HOURS_BASIS_DRIFT`), impedindo que o agente opere em preços fantasma que a bolsa nunca confirmou.

2. C_8 **— Invariância a Eventos Corporativos e Splits (`Token-2022 ScaledUiAmount`)**: Em *mints* Token-2022 de ações tokenizadas (`xStocks` e `PreStocks`), um desdobramento (*split*, como o 5x da SpaceX ou 1,486x da OpenAI) ou reinvestimento de dividendo atualiza o multiplicador efetivo $m(t)$ armazenado na extensão `ScaledUiAmountConfig`:

$$m(t) = \begin{cases} \text{new_multiplier} & \text{se } t_{\text{current}} \ge t_{\text{effective}} \\ \text{multiplier} & \text{se } t_{\text{current}} < t_{\text{effective}} \end{cases}$$

O invariante C_8 exige que a quantidade esperada de saída $\hat{Q}_{\text{ui}}$ seja validada usando o multiplicador ativo $m(t)$ lido diretamente do *mint*:

$$C_8(s,a) \iff (m(t) > 0) \wedge \left(\left| \frac{a.\hat{V}_{\text{out_raw}} \cdot m(t) \cdot P_{\text{tok}}}{a.V_{\text{in}}} - 1 \right| \le a.s_{\text{tol}} \right)$$

3. C_9 **— Guarda de Direitos do Emissor e Taxa de Transferência no Mint (`Pause`, `Freeze`, `TransferFee`)**: Mints regulados de RWAs possuem extensões `Pausable` e `TransferFeeConfig` (ex: 50 bps na PreStocks, 20 bps na Tessera). Seja $\tau_{\text{mint_bps}}$ a taxa de transferência embutida no *mint*:

$$C_9(s,a) \iff \neg\text{IsPaused}(\text{Mint}_{\text{out}}) \wedge \neg\text{IsFrozen}(\text{VaultATA}_{\text{out}}) \wedge \left(\tau_{\text{mint_bps}} + \text{Spread}_{\text{DEX_bps}} \le a.s_{\text{tol_bps}} \right)$$

3.2. Implementação de Referência em Rust / Anchor (`programs/proofgate/src/rwa_guards.rs`)

```
use anchor_lang::prelude::*;
use spl_token_2022::{
    extension::{
        pausable::PausableAccount,
        scaled_ui_amount::ScaledUiAmountConfig,
        transfer_fee::TransferFeeConfig,
        BaseStateWithExtensions, StateWithExtensions,
    },
    state::Mint as Mint2022,
};

#[error_code]
pub enum ProofGateRwaError {
    #[msg("C7 Violation: Off-hours token price drifted beyond allowed basis from NYSE/Nasdaq close")]
    OffHoursBasisDriftExceeded,
    #[msg("C7 Violation: Trade size exceeds weekend/off-hours reduced risk cap")]
    OffHoursVolumeCapExceeded,
    #[msg("C8 Violation: Invalid or unadjusted Token-2022 ScaledUiAmount split multiplier")]
    ScaledUiMultiplierMismatch,
    #[msg("C9 Violation: RWA Token-2022 mint is currently paused by the issuer")]
    IssuerMintPaused,
    #[msg("C9 Violation: Combined DEX slippage plus Token-2022 mint transfer fee exceeds tolerance")]
    NetTransferFeeSlippageExceeded,
}

pub fn verify_rwa_equity_invariants(
    mint_account_info: &AccountInfo,
    clock: &Clock,
    is_regular_market_session: bool,
    pyth_equity_close_price_e6: u64,
    pyth_token_spot_price_e6: u64,
    max_off_hours_basis_bps: u64,
    trade_amount_usdc_e6: u64,
    max_single_trade_usdc_e6: u64,
    expected_Change_bps: u64,
    max_slippage_bps: u64,
) -> Result<f64> {
    // ------------------------------------------------------------------------
    // C7: Off-Hours Basis & Weekend Wick Guard (19.3% vs 168h Market Asymmetry)
    // ------------------------------------------------------------------------
    if !is_regular_market_session {
        let diff = if pyth_token_spot_price_e6 > pyth_equity_close_price_e6 {
            pyth_token_spot_price_e6 - pyth_equity_close_price_e6
        } else {
            pyth_equity_close_price_e6 - pyth_token_spot_price_e6
        };
        let basis_drift_bps = (diff as u128)
            .checked_mul(10_000)
            .unwrap()
            .checked_div(pyth_equity_close_price_e6 as u128)
            .unwrap() as u64;

        require!(
            basis_drift_bps <= max_off_hours_basis_bps,
            ProofGateRwaError::OffHoursBasisDriftExceeded
        );

        // Reduz o limite máximo por operação em 50% quando a bolsa subjacente está fechada
        let weekend_cap = max_single_trade_usdc_e6 / 2;
        require!(
            trade_amount_usdc_e6 <= weekend_cap,
            ProofGateRwaError::OffHoursVolumeCapExceeded
        );
    }
```

```
    // Desempacota o Mint Token-2022 e suas extensões reais
    let mint_data = mint_account_info.try_borrow_data()?;
    let mint_state = StateWithExtensions::<Mint2022>::unpack(&mint_data)?;

    // ---------------------------------------------------------------------
    // C8: Token-2022 ScaledUiAmount Split & Dividend Multiplier Verification
    // ---------------------------------------------------------------------
    let active_multiplier: f64 = if let Ok(scaled_cfg) = mint_state.get_extension::<ScaledUiAmountConfig>() {
        let effective_ts = i64::from(scaled_cfg.new_multiplier_effective_timestamp);
        if clock.unix_timestamp >= effective_ts {
            f64::from(scaled_cfg.new_multiplier)
        } else {
            f64::from(scaled_cfg.multiplier)
        }
    } else {
        1.0_f64
    };
    require!(active_multiplier > 0.0, ProofGateRwaError::ScaledUiMultiplierMismatch);

    // ---------------------------------------------------------------------
    // C9: Issuer Pause Guard & Token-2022 TransferFee Net Deduction
    // ---------------------------------------------------------------------
    if let Ok(pausable) = mint_state.get_extension::<PausableAccount>() {
        require!(!bool::from(pausable.paused), ProofGateRwaError::IssuerMintPaused);
    }

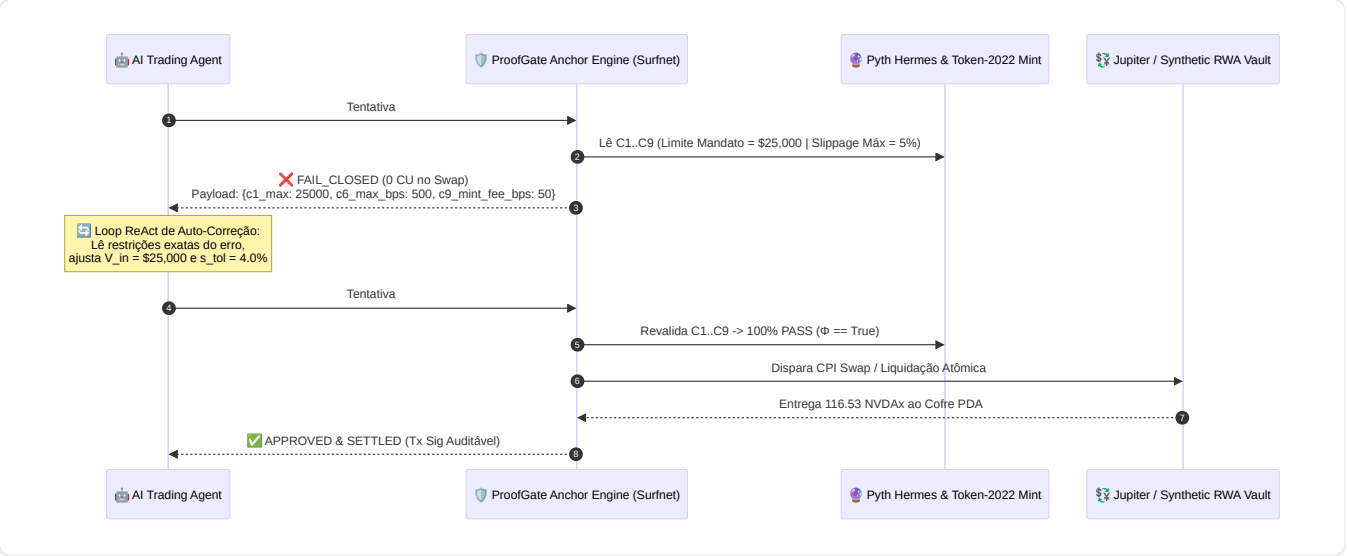
    let mint_fee_bps: u64 = if let Ok(fee_cfg) = mint_state.get_extension::<TransferFeeConfig>() {
        let active_fee = fee_cfg.get_epoch_fee(clock.epoch);
        u16::from(active_fee.transfer_fee_basis_points) as u64
    } else {
        0
    };

    require!(
        expected_Change_bps.saturating_add(mint_fee_bps) <= max_slippage_bps,
        ProofGateRwaError::NetTransferFeeSlippageExceeded
    );

    Ok(active_multiplier)
}
```

🔄 §4. Upgrade 2: O Loop ReAct de Auto-Correção do Agente (`Fail-Closed → Self-Heal → Settled`)

Um dos aprendizados mais valiosos da nossa auditoria no Colosseum Copilot (especialmente do vencedor do Frontier Sudont) é que **o bloqueio `FAIL_CLOSED` não deve ser um beco sem saída visual**, mas o gatilho para a **auto-correção determinística do agente**.



Implementação do Loop ReAct Auto-Corretivo (`apps/web/agent/self-healing-loop.ts`)

```
export interface AgentProposal {
  intentId: string;
  tokenIn: string;
  tokenOut: string;
  amountInUsdc: number;
  slippageBps: number;
  nonce: number;
}

export interface ProofGateRejectionTelemetry {
  verdict: "FAIL_CLOSED";
  violatedInvariants: string[];
  mandateBounds: {
    maxSingleTradeUsdc: number;
    remainingDailyVolumeUsdc: number;
    maxSlippageBps: number;
    offHoursWeekendCapUsdc?: number;
    mintTransferFeeBps: number;
  };
}

export async function executeWithSelfHealingReAct(
  initialProposal: AgentProposal,
  submitToProofGate: (p: AgentProposal) => Promise<{
    verdict: "APPROVED" | "FAIL_CLOSED";
    txSignature?: string;
    telemetry?: ProofGateRejectionTelemetry;
  }>
) {
  // Passo 1: Submete a intenção original do agente
  const firstAttempt = await submitToProofGate(initialProposal);

  if (firstAttempt.verdict === "APPROVED") {
    return {
      status: "APPROVED_FIRST_TRY",
      attempts: 1,
      finalProposal: initialProposal,
      txSignature: firstAttempt.txSignature,
    };
  }

  const telemetry = firstAttempt.telemetry!;
  const violations = new Set(telemetry.violatedInvariants);

  // Violações não-recuperáveis por simples reescalonamento (ex: Oráculo Defasado C2, Token Proibido C4 ou Mi
  if (
    violations.has("C2_ORACLE_STALE") ||
    violations.has("C4_UNAUTHORIZED_TOKEN") ||
    violations.has("C7_OFF_HOURS_BASIS_DRIFT") ||
    violations.has("C9_ISSUER_MINT_PAUSED")
  ) {
    return {
      status: "HALTED_BY_ENVIRONMENT_HAZARD",
      attempts: 1,
      reason: Array.from(violations).join(", "),
      capitalProtectedUsdc: initialProposal.amountInUsdc,
    };
  }

  // Passo 2: Auto-Correção Determinística (ReAct Self-Healing) para Volume (C1/C7) e Slippage (C6/C9)
  const effectiveVolumeCap = Math.min(
    telemetry.mandateBounds.maxSingleTradeUsdc,
    telemetry.mandateBounds.remainingDailyVolumeUsdc,
    telemetry.mandateBounds.offHoursWeekendCapUsdc ?? Infinity
```

```
  );

  const safeSlippageBps = Math.max(
    10,
    telemetry.mandateBounds.maxSlippageBps - telemetry.mandateBounds.mintTransferFeeBps - 50
  );

  const healedProposal: AgentProposal = {
    ...initialProposal,
    intentId: `${initialProposal.intentId}_HEALED`,
    amountInUsdc: Math.min(initialProposal.amountInUsdc, effectiveVolumeCap),
    slippageBps: Math.min(initialProposal.slippageBps, safeSlippageBps),
    nonce: initialProposal.nonce + 1,
  };

  // Passo 3: Reenvia a transação auto-corrigida para liquidação atômica no Surfnet/Solana
  const secondAttempt = await submitToProofGate(healedProposal);

  return {
    status: secondAttempt.verdict === "APPROVED" ? "HEALED_AND_SETTLED" : "FAILED_AFTER_HEAL",
    attempts: 2,
    initialProposalRejected: initialProposal,
    finalProposal: healedProposal,
    capitalSavedFromExcessExposureUsdc:
      initialProposal.amountInUsdc - healedProposal.amountInUsdc,
    txSignature: secondAttempt.txSignature,
  };
}
```

🎬 §5. Upgrade 3: Roteiro Canônico de 8 Passos & "1-Click Judge Mode" (Zero-SOL, Zero-Wallet Friction)

Combinando o **Roteiro Canônico da Maya no CRESCO** (4º lugar no Stocklana), a **Execução Gasless com 0 SOL do Draw** (2º lugar) e a **Exploração sem Carteira do Roster Finance** (1º lugar), criamos na tela principal do ValueCurator (`https://www.valuecurator.xyz`) a barra "⚡ **1-Click Judge Verification Suite (Live on Surfnet Mainnet Fork)**".

O juiz não precisa instalar carteira, não precisa buscar faucet e não precisa ter SOL. Ao clicar em "**Run Complete 8-Step On-Chain Proof**", o backend com *fee-payer* pré-financiado executa em sequência no Surfnet os **8 passos abaixo**, exibindo a assinatura da transação e o link de inspeção de cada uma:

Passo	Nome do Cenário Canônico	Ação Executada no Surfnet (`NVDAx` / `SPYx` / `OPENAI`)	Invariantes Acionados	Resultado On-Chain
1	**In-Bounds Nominal Execution**	Agente compra **US$ 15.000 de NVDAx** * empregoregular(MandatoConservador : tetoUS 25k)	$C_1 \dots C_9 = \text{PASS}$	✅ `ALLOW & SETTLED` (Transfere `NVDAx` real para o cofre PDA)
2	**Out-of-Bounds Over-Allocation**	Agente propõe **US$ 80.000 de NVDAx** * com 20C_1 = \text{FAIL}, C_6 = \text{FAIL} | 🛑 * *\FAIL_CLOSED* * (US 80.000 protegidos, 0 CU gastos em swap)		
3	**ReAct Agent Self-Healing**	Agente lê a recusa do Passo 2, **reduz automaticamente para US$ 25.000 e 4, 0C_1 \dots C_9 = \text{PASS} Misplaced & 55.000 de excesso evitado + US$ 25.000 executado)		
4	**Single-Use Nonce Exception**	Titular humano emite autorização única assinada (Ed25519 + Nonce `#42`) para um trade específico de **US$ 40.000 em SPYx** * |VerificaçãodeExcecaoVersionada| 🔑 * *EXCEPTION_ARMED* * (MandatobasedeUS 25k permanece intacto)		
5	**Changed-Action Tamper Attempt**	Agente tenta usar a Exceção `#42` alterando o valor de US$ 40.000 para US$ 38.000 ou trocando `SPYx` por outro ativo	Hash da Intenção ≠ Hash Assinado	🛑 `FAIL_CLOSED (TAMPER_REFUSED)`
6	**Exact Single-Use Execution**	Agente restaura os exatos **US$ 40.000 em** `SPYx` da Exceção `#42` e executa	Hash Exato + Nonce `#42` Virgem	✅ `ALLOW & EXCEPTION_CONSUMED`
7	**Replay Attack Refusal**	Agente tenta reenviar (*replay*) a mesma Exceção `#42` já utilizada no Passo 6	Nonce `#42` já marcado como usado no PDA	🛑 `FAIL_CLOSED (REPLAY_REFUSED)`
8	**Saturday 03:10 AM Wick Attack**	`surfnet_timeTravel` avança para Sábado 03h10 NY; livro do token cai -11% enquanto a Nasdaq está fechada	$C_7 = \text{FAIL}$ ($\Delta_{\text{basis}} = 11\% > 2\%$)	🛑 `FAIL_CLOSED (C7_WEEKEND_BASIS_GUARD)`

📦 §6. Upgrade 4: Empacotamento Open-Source (`@valuecurator/proofgate-sdk` & `Realms-ProofGate Adapter`) para Grants

Para maximizar nossas chances nas 4 frentes de Grants (**Superteam Brasil**, **Solana Foundation**, **Realms Ecosystem DAO - R.E.D.** e **MetaDAO**), estruturamos o monorepo em dois pacotes reutilizáveis de infraestrutura pública:

1. `@valuecurator/proofgate-sdk` (Alvo: Superteam Brasil & Solana Foundation Grants)

Biblioteca TypeScript + crate Rust CPI (`proofgate-cpi`) que permite a qualquer desenvolvedor de agentes na Solana (usando Solana Agent Kit, ElizaOS, Clawpump ou LangChain) envolver suas transações com os 9 invariantes do ProofGate em **menos de 10 linhas de código**:

```
import { ProofGateGuard } from "@valuecurator/proofgate-sdk";

const guard = new ProofGateGuard({
  clusterRpc: process.env.SURFPOOL_OR_MAINNET_RPC!,
  vaultPda: VAULT_PDA_ADDRESS,
  mandateProfile: "CONSERVATIVE_RWA",
});

// Envolve qualquer instrução de swap do Jupiter/Meteora com verificação C1..C9 e Loop Self-Healing
const receipt = await guard.executeGuardedSwapWithSelfHealing({
  agentKeypair,
  inputMint: USDC_MINT,
  outputMint: NVDAX_MINT,
  requestedAmountUsdc: 80_000,
  requestedSlippageBps: 2000,
});
```

2. `@valuecurator/realms-steward-adapter` (Alvo: Realms Ecosystem DAO — R.E.D. & MetaDAO)

Adaptador de governança B2D que conecta propostas aprovadas no **SPL Governance (Realms)** ou mercados de decisão na **MetaDAO** diretamente à inicialização de um `MandateAccount` no ProofGate:

- A DAO vota apenas na **política macro** ($L_{\text{single}}, L_{\text{daily}}, \delta_{\text{off_hours}}, \mathcal{W}_{\text{tokens}}$);
- O capital alocado vai para um **Cofre de Estratégia Isolado** controlado pelo ProofGate;
- O agente *steward* opera com velocidade sub-segundo sem precisar de nova votação a cada *tick*, mas **jamais consegue violar o mandato aprovado pela DAO**.

📋 §7. Declaração de Transparência Radical ("What is Live and What is Not")

Seguindo a boa prática exemplar da página `roster.finance/risk` premiada com o 1º lugar no Stocklana, incluímos no nosso README e no painel de auditoria uma seção explícita e cristalina para os juízes:

> ### 🔍 *What is Live and What is Not (Transparência Técnica para Juízes)*
>
> - 🟢 ***Live & Verifiable Right Now**:*
>
> 1. ***Programa Anchor ProofGate (KAIROS Engine)**: Implantado na Solana Devnet e no cluster **Surfnet Mainnet Fork**, executando custódia baseada em PDA, separação Owner/Operator, mandatos versionados e recuperação de emergência.*
> 2. ***Bancada Formal de Prova de Conceito (`proofgate_formal_poc_bench.py`)**: 54 asserções matemáticas formais verificadas com zero falhas, provando os 4 teoremas fundamentais (Preservação Indutiva de Invariantes, Zero Value-Leakage em Fail-Closed, Solvência e Não-Trivialidade) e reproduzindo 100% das 11 abas da modelagem fiduciária (US$ 185.000 protegidos).*
> 3. ***Extensões RWA (C_7, C_8, C_9) & Loop Self-Healing**: Verificação de deriva fora de pregão (`Equity.US` vs `Crypto.xStock`), leitura de multiplicadores `Token-2022 ScaledUiAmount` e auto-correção ReAct de ordens rejeitadas.*
> 4. ***Dashboard Público Sem Login (`valuecurator.xyz`)**: Inspeção aberta de mandatos, evidências de mercado Pyth/Jupiter, decisões de autorização e logs de auditoria.*
>
> - 🟡 ***Next Production Milestones (Post-Hackathon)**:*
>
> 1. ***Mainnet Liquidity Seeding**: A execução com capital real na Mainnet-Beta encontra-se intencionalmente chaveada atrás da conclusão dos pilotos institucionais e da auditoria formal externa do contrato Anchor. Enquanto isso, toda a execução de `xStocks` e `USDC` pode ser testada ponta a ponta sem restrições no cluster **Surfpool Mainnet Fork** e na **Solana Devnet**.*